# PYSPARK GROUPBY AND AGGREGATIONS

# groupBy in Spark

- 1. **groupBy()** –In Spark, the **`groupBy`** method is used to group data in a DataFrame based on one or more columns, similar to the `GROUP BY` clause in SQL. This is often followed by aggregation operations to compute summary statistics for each group.

**1.Single Column GroupBy**:

- Grouping by a single column and applying an aggregation function.

```
from pyspark.sql.functions import avg, sum, count
# Sample DataFrame
data = [(1, "Alice", "HR", 5000),(2, "Bob", "IT", 6000),(3, "Charlie", "Finance", 7000),(4, "David", "IT", 8000)]
columns = ["id", "name", "department", "salary"]
df = spark.createDataFrame(data, columns)
# Group by department and calculate average salary
df_grouped = df.groupBy("department").agg(
    avg("salary").alias("avg_salary")
)
display(df_grouped)
```

```
▸ (2) Spark Jobs
▸ df:  pyspark.sql.dataframe.DataFrame = [id: long, name: string ... 2 more fields]
▸ df_grouped:  pyspark.sql.dataframe.DataFrame = [department: string, avg_salary: double]
```

Table +

| | department | avg_salary |
|---|---|---|
| 1 | HR | 5000 |
| 2 | IT | 7000 |
| 3 | Finance | 7000 |

**In this example:**

- The DataFrame is grouped by the **`department`** column.
- The **`agg`** method is used to calculate the average salary for each department.
- The result is displayed using the **`display`** function.

# groupBy in Spark

**2.Multiple Columns GroupBy**:

- Grouping by multiple columns and applying aggregation functions.
- This allows you to perform aggregations on more granular levels of grouped data.

```
%python
# Group by department and name, then calculate sum and count of salary
df_grouped_multi = df.groupBy("department", "name").agg(
    sum("salary").alias("total_salary"),
    count("id").alias("count")
)
display(df_grouped_multi)
```

(2) Spark Jobs

df_grouped_multi: pyspark.sql.dataframe.DataFrame = [department: string, name: string ... 2 more fields]

Table

| | department | name | total_salary | count |
|---|---|---|---|---|
| 1 | HR | Alice | 5000 | 1 |
| 2 | IT | Bob | 6000 | 1 |
| 3 | Finance | Charlie | 7000 | 1 |
| 4 | IT | David | 8000 | 1 |

**In this example:**

- The DataFrame is grouped by both the **`department`** and **`name`** columns.
- The **`agg`** method is used to calculate the total salary and the count of employees for each department and name combination.
- The result is displayed using the `display` function.

# Aggregations in Spark

- In PySpark, you can perform single and multiple aggregations using the `groupBy` method followed by aggregation functions

**1.Single Aggregation**:

- For a single aggregation, you can directly use aggregation functions like `count`, `sum`, or `avg` after `groupBy`. This is useful when you want to perform one specific aggregation on the grouped data.

```
%python
from pyspark.sql.functions import avg
# Group by department_id and calculate the average salary
final_df=emp_df.groupBy("department_id").avg("salary")
'''Alternative==
final_df = emp_df.groupBy("department_id").agg(avg("salary").alias
("avg_salary"))'''
# Display the final DataFrame
final_df.show()
```

▸ (2) Spark Jobs

▸ final_df: pyspark.sql.dataframe.DataFrame = [department_id: integer, avg(salary): double]

```
+-------------+------------------+
|department_id|       avg(salary)|
+-------------+------------------+
|            1|504876.96401242825|
|            6|504428.12590014644|
|            3| 504697.6808514883|
|            5| 504167.9429997006|
|            9| 504945.3055672206|
|            4| 505419.4963977089|
|            8| 505299.1226286386|
```

**Note**:

- single aggregation can be achieved using the **agg** method, where you explicitly specify the aggregation function and column to aggregate.
- On the other hand, single aggregation can also be performed **without** using the **agg** method by directly applying aggregation functions like `avg`, `sum`, or `count` after the `groupBy` operation.
- Both methods allow you to calculate a single metric, such as average salary, for grouped data.

# Aggregations in Spark

- In PySpark, you can perform single and multiple aggregations using the `groupBy` method followed by aggregation functions

**2.Multiple Aggregations Using agg:**

- Multiple aggregation involves grouping data by one or more columns and applying multiple aggregation functions to the grouped data. This allows for a more comprehensive summary of the data with various metrics.

Just now (2s) 10

```
%python
from pyspark.sql.functions import avg,sum,count
# Group by department_id and calculate the average salary
final_df = emp_df.groupBy("department_id").agg(avg("salary").alias
("avg_salary"),sum("salary").alias("total_salary"),count("*").alias
("count"))
# Display the final DataFrame
final_df.show()
```

(2) Spark Jobs

final_df: pyspark.sql.dataframe.DataFrame = [department_id: integer, avg_salary: double ... 2 more fields]

| department_id | avg_salary | total_salary | count |
|---|---|---|---|
| 1 | 504876.96401242825 | 5.0210518948E10 | 99451 |
| 6 | 504428.12590014644 | 5.0294510721E10 | 99706 |
| 3 | 504697.6808514883 | 5.059493311E10 | 100248 |
| 5 | 504167.9429997006 | 5.052669568E10 | 100210 |
| 9 | 504945.3055672206 | 5.0501599791E10 | 100014 |
| 4 | 505419.4963977089 | 5.0650109412E10 | 100214 |
| 8 | 505299.1226286386 | 5.0740621997E10 | 100417 |
| 7 | 504514.38453985273 | 5.035058149E10 | 99805 |
| 10 | 502682.2575766687 | 5.0157635661E10 | 99780 |

**Common Aggregation Functions**

- **avg**: Computes the average of the values.
- **sum**: Computes the sum of the values.
- **count**: Counts the number of rows.
- **min**: Finds the minimum value.
- **max**: Finds the maximum value.

# SelectExpr in pyspark

- The `selectExpr` method in Apache Spark allows you to specify columns using SQL expressions. It is a variant of the **`select`** method that accepts SQL expressions and returns an updated DataFrame. This method is useful for performing transformations on columns using SQL expressions directly.

Just now (2s) 13

```
emp_df.selectExpr("upper(first_name) as first_name","lower(last_name)
as last_name","concat(first_name,' ',last_name) as full_name" ,
"salary").distinct().display()
```

(2) Spark Jobs

Table +

| | first_name | last_name | full_name | 1.2 salary |
|---|---|---|---|---|
| 1 | MARK | webster | Mark Webster | 522518 |
| 2 | DIANA | savage | Diana Savage | 76484 |
| 3 | THOMAS | gillespie | Thomas Gillespie | 868882 |
| 4 | RYAN | thornton | Ryan Thornton | 28396 |
| 5 | DANA | adams | Dana Adams | 65002 |
| 6 | CHRISTOPHER | haley | Christopher Haley | 325425 |
| 7 | KIMBERLY | wells | Kimberly Wells | 789680 |
| 8 | MATTHEW | hogan | Matthew Hogan | 425189 |
| 9 | SAMANTHA | cain | Samantha Cain | 608320 |
| 10 | KEITH | buchanan | Keith Buchanan | 46308 |
| 11 | JANICE | robertson | Janice Robertson | 88777 |

- **`upper(first_name) as first_name`:**
  - Converts the `first_name` column to uppercase.
  - Renames the resulting column to `first_name`.
- **`lower(last_name) as last_name`:**
  - Converts the `last_name` column to lowercase.
  - Renames the resulting column to `last_name`.
- **`concat(first_name, ' ', last_name) as full_name`:**
  - Concatenates the `first_name` and `last_name` columns with a space in between.
  - Renames the resulting column to `full_name`.
- **`distinct()`:**
  - Removes duplicate rows from the resulting DataFrame.

# Top Interview-style Questions

1. 💬 What is the difference between withColumn() and select()?
2. 💬 How does when() differ from a traditional if-else in Python?
3. 💬 What happens if you apply dropna() without any parameters?
4. 💬 How does isin() work internally in Spark?
5. 💬 Can you combine multiple transformations in one chain? Show an example.
6. 💬 What's the difference between groupBy().agg() and selectExpr() with aggregations?
7. 💬 Why should we use lit() when adding constants?
8. 💬 Is filter() and where() the same in PySpark? Any performance difference?

🧠 **Tip**: The real power of PySpark comes from chaining multiple transformations efficiently. Practice building pipelines step-by-step.